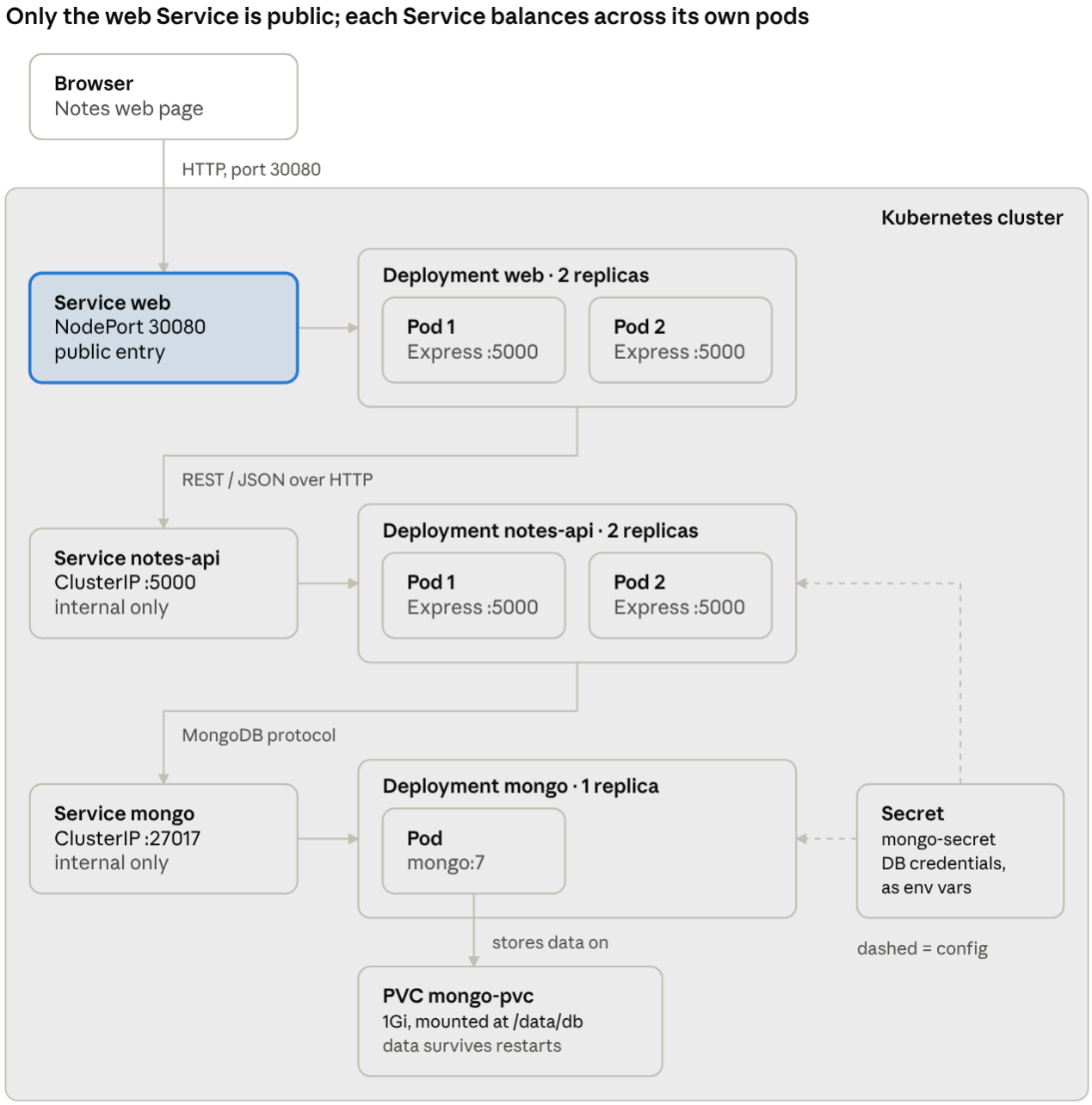
Architecture Design

The Notes application is split into a public gateway (`web`), an internal data service (`notes-api`) and a database (`mongo`), each running as its own Kubernetes Deployment behind its own Service.

Architecture at a glance



component view · 3 Services, 3 Deployments, Secret, PVC

Each tier is a Service in front of a Deployment. A request goes from the browser through each Service to one of its pods, then to the next tier. The Secret supplies the database credentials to `notes-api` and `mongo`.

Components and responsibilities

Component	Role	Responsibilities
User interface	What the user sees	Single HTML page with plain JavaScript. Calls `/api/*` on `web` with `fetch`, shows notes, stats and the serving pod names. Inserts note text with `textContent` so notes cannot inject HTML.
Gateway / BFF (`web`)	The only public entry point	Serves the user interface. Exposes the public REST API. Forwards note requests to `notes-api` over REST with a 3-second timeout and returns `503` if it is unavailable. Computes `/api/stats` from the data it receives. Holds no data.
Notes service (`notes-api`)	Owner of the notes data	Exposes the internal REST API for notes. Validates input (1–500 characters, valid ids). Reads and writes MongoDB. Returns proper status codes (`201`, `400`, `404`, `500`).
Database (`mongo`)	Durable storage	Stores the `notes` collection in the `notesdb` database. Requires a username and password. Writes its data to a persistent volume.
Configuration and secrets	Settings kept outside the code	A Kubernetes Secret holds the database credentials. Environment variables hold service URLs (`MONGO_URL`, `NOTES_API_URL`).
Kubernetes platform	Runs and connects everything	Schedules pods, restarts failed ones, gives each Service a DNS name, load-balances requests across replicas and only sends traffic to pods that pass their readiness check.

Mapping: components to microservices

Three microservices implement the six components: two built by me, plus the official MongoDB image.

Software component	Microservice	Container image	Kubernetes objects	Source
User interface	`web`	`<user>/notes-web:1.0` (built by me)	served by the `web` Deployment	`web/static/index.html`
Gateway / BFF	`web`	`<user>/notes-web:1.0` (built by me)	Deployment `web`, Service `web` (NodePort 30080)	`web/server.js`
Notes service	`notes-api`	`<user>/notes-api:1.0` (built by me)	Deployment `notes-api`, Service `notes-api` (ClusterIP)	`notes-api/server.js`
Database	`mongo`	`mongo:7` (official)	Deployment `mongo`, Service `mongo` (ClusterIP), PersistentVolumeClaim `mongo-pvc`	`k8s/app.yaml`
Configuration and secrets	used by `notes-api` and `mongo`	none	Secret `mongo-secret`, environment variables	`k8s/app.yaml`
Kubernetes platform	none (runs all three)	none	all of the above	`k8s/app.yaml`

`<user>` is the Docker Hub account the images are pushed to.

Architecture principles and patterns

Principle or pattern	What it means	How it is applied here
Microservices, single responsibility	Each service does one job and is deployed on its own	`web` handles presentation and the public API; `notes-api` handles notes data. Separate code, images and Deployments.
API Gateway / Backend for Frontend	One public entry point that shields internal services	The browser only talks to `web`. `notes-api` and `mongo` are ClusterIP Services, unreachable from outside.

Principle or pattern	What it means	How it is applied here
Database per service	Only the owning service touches its data	Only `notes-api` has the database credentials and connects to MongoDB. `web` gets data through the REST API.
Synchronous REST communication	Services talk over HTTP with JSON	`web` calls `GET/POST /notes` and `DELETE /notes/{id}` on `notes-api` with Node.js `fetch`.
Stateless services, horizontal scaling	Any replica can answer any request	`web` and `notes-api` keep no local state, so each Deployment can be scaled on its own with `kubectl scale`.
Service discovery and load balancing	Services find each other by name; traffic is spread	`web` calls `http://notes-api:5000`; `notes-api` calls `mongo:27017`. Kubernetes DNS resolves the names and each Service balances across its pods.
Health endpoint monitoring	Each service reports whether it is ready	`GET /health` on both services, used as readiness probes.
Fail fast (timeout + fallback)	A slow dependency must not hang the caller	`web` gives up on `notes-api` after 3 seconds and returns `503 notes-api unavailable`.
External configuration	Settings live outside the code	Service URLs come from environment variables; credentials from a Kubernetes Secret.
Persistent storage for stateful parts	Data outlives the containers	MongoDB data lives on a PersistentVolumeClaim. The `Recreate` strategy stops two database pods writing the same volume.
Declarative infrastructure	The desired state is described in a file	All objects are in `k8s/app.yaml` and are created with one `kubectl apply`.

How the components interact

Adding a note passes through every component:

1. The user types a note and clicks **Add**. The page sends `POST /api/notes` with `{"text": "..."}` to port 30080.
2. The `web` Service picks one of the `web` pods.
3. That `web` pod calls `POST http://notes-api:5000/notes` with the same body.
4. The `notes-api` Service picks one of the `notes-api` pods.
5. `notes-api` checks the text is 1–500 characters. If not, it returns `400` and nothing is stored.
6. `notes-api` inserts the note into MongoDB through the `mongo` Service, using the credentials from the Secret. MongoDB writes it to the persistent volume.
7. `notes-api` returns `201` with the new note's id, plus an `X-Served-By` header naming its pod.
8. `web` relays the response to the browser and adds its own pod name. The page reloads the list and calls `GET /api/stats`, which `web` computes from `GET /notes`.

Listing and deleting notes follow the same path. If `notes-api` is down or slower than 3 seconds, `web` stops at step 3 and returns `503`.

Deployment details

All values come from `k8s/app.yaml`.

	`web`	`notes-api`	`mongo`
Image	`<user>/notes-web:1.0`	`<user>/notes-api:1.0`	`mongo:7`
Replicas	2 (scalable)	2 (scalable)	1 (not scaled)
Container port	5000	5000	27017
Service type	NodePort, port 80 → 5000, node port 30080	ClusterIP, port 5000	ClusterIP, port 27017
Reachable from	Outside the cluster	Inside the cluster only	Inside the cluster only
Readiness probe	`GET /health`	`GET /health`	none
CPU request / limit	50m / 250m	50m / 250m	not set
Memory request / limit	64Mi / 128Mi	64Mi / 128Mi	not set

	`web`	`notes-api`	`mongo`
Configuration	`NOTES_API_URL`	`MONGO_URL` built from `mongo-secret`	root user from `mongo-secret`
Storage	none	none	`mongo-pvc`, 1Gi, ReadWriteOnce, at `/data/db`
Update strategy	Rolling update	Rolling update	Recreate

Both Node.js images use `node:20-alpine`, install dependencies with `npm ci --omit=dev` and run as the non-root `node` user.